



Orientação a Objetos

Introdução

O que é Orientação a Objetos?

- Orientação a Objetos (OOP) é um **paradigma de programação** que organiza o software em torno de objetos;
- Esses objetos **representam entidades do mundo real** e interagem entre si para resolver problemas;
- **Cada objeto é uma instância de uma classe**, que define os atributos e comportamentos do objeto;
- OOP facilita o **design modular** do software, tornando-o mais fácil de manter, reutilizar e expandir;
- Os principais pilares da OOP são **abstração, encapsulamento, herança e polimorfismo**;
- Benefícios da Orientação a Objetos
 - **Modularidade:** O código é organizado em classes e objetos, tornando-o mais fácil de entender e manter;
 - **Reuso de Código:** A herança e a modularidade facilitam a reutilização de partes do código em diferentes aplicações;
 - **Facilidade de Manutenção:** Alterações podem ser feitas em uma classe sem afetar outras partes do sistema, promovendo maior flexibilidade;
 - **Segurança:** O encapsulamento protege os dados e controla o acesso aos atributos e métodos de uma classe;



Terminologia de OO

- **Classe**
 - Um "molde" ou "blueprint" para criar objetos;
 - Define os atributos (propriedades) e métodos (comportamentos) de um objeto;
- **Objeto**
 - Instância de uma classe;
 - Exemplo: Um carro é um objeto da classe "Carro", com atributos como cor, modelo e métodos como acelerar, frear;
- **Atributo (ou Propriedade)**
 - São variáveis dentro da classe que armazenam os dados do objeto;
 - Exemplo: No objeto "Carro", atributos podem ser cor, modelo, ano;
- **Método**
 - Função definida dentro de uma classe que representa uma ação ou comportamento do objeto;
 - Exemplo: No objeto "Carro", métodos podem ser acelerar(), frear();
- **Instanciação**
 - Processo de criar um novo objeto a partir de uma classe;
 - Utiliza-se a palavra-chave new em Java para instanciar um objeto;



Criando uma classe

- Uma classe é um **modelo** que define as **propriedades** e **comportamentos** de um objeto;
- Classes são criadas usando a **palavra-chave** **class**;
- O nome da classe deve ser um substantivo e **começar com letra maiúscula**;
- Dentro de uma classe, você pode definir **atributos (propriedades)** e **métodos (comportamentos)**;
- Uma classe não faz nada por si só até que seja **instanciada** (criação de um objeto);



Instanciando objetos

- Objetos são **instâncias de uma classe**;
- Para instanciar um objeto, usamos a **palavra-chave new**;
- A instância **cria uma cópia do modelo** da classe em memória;
- Cada objeto **tem seus próprios valores de atributos**, independentes de outros objetos;
- A instância de um objeto **permite acessar métodos e atributos** definidos na classe;
- A sintaxe básica para criar um objeto é:
 - `NomeDaClasse nomeDoObjeto = new NomeDaClasse();`
- Ao instanciar um objeto, podemos acessar seus métodos e definir valores para seus atributos;



Criando métodos

- Métodos **são funções dentro de uma classe** que definem o comportamento de um objeto;
- Eles permitem que objetos realizem **ações ou operações**;
- A sintaxe de um método inclui o **tipo de retorno, o nome do método, e os parâmetros**;
- **Métodos podem ou não retornar um valor**, dependendo da definição;
- Para criar um método, usamos a seguinte estrutura:

```
public TipoDeRetorno nomeDoMetodo(TipoParametro parametro) {  
    // Corpo do método  
}
```

- Se o método não retornar valor, **usamos void**;
- Os métodos **são chamados a partir de uma instância de um objeto** da classe;
- Os métodos podem receber parâmetros para trabalhar com dados e retornar resultados;



O que é Encapsulation?

- **Encapsulamento (encapsulation)** é um dos pilares da Programação Orientada a Objetos (OOP);
- Ele consiste em **esconder os detalhes internos de uma classe** e expor apenas o necessário para o usuário da classe;
- A ideia principal é **proteger os dados de acessos indevidos** ou modificações diretas;
- Encapsulamento é obtido através do uso de modificadores de acesso, como **private**, **protected**, e **public**;
- Propriedades privadas só podem ser acessadas ou modificadas por métodos específicos da classe (**getters e setters**);
- Getters e setters fornecem uma maneira controlada de **acessar e modificar os atributos da classe**;
- Encapsulamento promove a integridade dos dados e facilita a manutenção do código;
- O objetivo é garantir que os atributos da classe só sejam alterados de maneira controlada e válida;



Criando propriedades

- Propriedades são atributos de uma classe que **definem as características de um objeto**;
- **Cada propriedade tem um tipo de dado**, como int, double, String, entre outros;
- As propriedades podem ser **públicas, privadas ou protegidas**, dependendo da visibilidade desejada;
- Propriedades **geralmente são declaradas no início da classe**;
- Propriedades públicas **podem ser acessadas diretamente**, enquanto propriedades privadas exigem métodos getters e setters;
- O valor das propriedades pode ser alterado diretamente (se público) ou por meio de métodos (se privado);
- Boas práticas de encapsulamento **sugerem o uso de propriedades privadas com métodos públicos** para acessar e modificar os valores;



O this em objetos

- O this é uma **palavra-chave em Java usada dentro de métodos** de uma classe para se referir ao objeto atual;
- Ele é útil para **distinguir entre os atributos do objeto e os parâmetros do método** com o mesmo nome;
- O this é **frequentemente usado em setters e construtores** para referenciar os atributos do objeto;
- Sempre que um método de uma classe é chamado, **o this se refere à instância do objeto** que fez a chamada;
- O this também pode ser usado para encadear métodos (method chaining) ou chamar outros construtores;



O que são Setters?

- Setters são métodos usados para **alterar os valores das propriedades** privadas de uma classe;
- Eles permitem o controle sobre **como os atributos de um objeto são modificados**;
- Setters seguem a convenção de nomeação: `setNomePropriedade()`;
- Setters **garantem que os valores dos atributos sejam válidos** antes de serem atribuídos;
- Com setters, é possível adicionar **lógica de validação ou manipulação** dos dados antes de atualizar o valor;



O que são Getters?

- Getters são métodos usados para **acessar os valores das propriedades privadas** de uma classe;
- **Eles permitem que outras classes ou métodos leiam os atributos**, mas sem permitir modificá-los diretamente;
- Getters seguem a convenção de nomeação: `getNomePropriedade()`;
- Um getter **retorna o valor da propriedade correspondente** e não deve alterar o estado do objeto;
- É uma prática comum usar getters para manter o **princípio do encapsulamento**;
- Getters ajudam a garantir que os atributos de um objeto sejam acessados de forma controlada;



Lógica no Getter e Setter

- Getters e Setters podem **conter lógica para validar, processar ou modificar os dados** antes de serem lidos ou gravados;
- A lógica nos Setters permite **validar os dados antes de armazená-los**, garantindo integridade;
- A lógica nos Getters pode **processar ou formatar os dados** antes de retorná-los;
- Isso melhora a segurança e o controle sobre como os atributos são manipulados;
- Alterar dados com lógica nos Getters e Setters **mantém o princípio do encapsulamento** e previne valores inválidos;



Método dentro de método

- Em Java, **métodos podem chamar outros métodos** dentro da mesma classe;
- Essa prática ajuda a modularizar o código, **quebrando funcionalidades complexas em partes menores**;
- Chamar um método dentro de outro **promove a reutilização de código** e facilita a manutenção;
- Ao combinar métodos, **é possível realizar ações mais complexas** e evitar duplicação de código;
- Métodos dentro de métodos ajudam a simplificar a lógica, tornando o código mais claro e legível;



O que são constructors?

- Construtores (constructors) são **métodos especiais usados para inicializar objetos**;
- Eles são **chamados automaticamente** quando um novo objeto é criado;
- O nome de um construtor deve ser o mesmo que o nome da classe;
- Construtores **não têm um tipo de retorno** (nem mesmo void);
- Podem ser usados para **atribuir valores iniciais aos atributos de um objeto**;
- Java fornece um construtor padrão se nenhum for definido, mas construtores personalizados permitem maior controle na criação do objeto;



Exercícios

- Vamos agora fazer os exercícios da seção de POO!



Modificadores de acesso

- Os modificadores de acesso **controlam a visibilidade dos membros de uma classe** (atributos e métodos);
- **public**: O membro pode ser acessado de qualquer lugar (dentro ou fora do pacote);
- **private**: O membro só pode ser acessado dentro da própria classe;
- **protected**: O membro pode ser acessado dentro da classe, suas subclasses e classes do mesmo pacote;
- O uso correto dos modificadores de acesso é fundamental **para aplicar o encapsulamento**;
- Eles protegem os dados e métodos, garantindo o controle de como eles são acessados e modificados;



Classes imutáveis

- Uma classe imutável é aquela cujas instâncias (objetos) **não podem ser modificadas depois de criadas**;
- Todos os atributos de uma classe imutável **são declarados como private e final**;
- **Não há setters**, e qualquer alteração nos atributos requer a criação de um novo objeto;
- Classes imutáveis garantem consistência e segurança no código, evitando mudanças inesperadas no estado do objeto;
- Exemplos de classes imutáveis nativas em Java incluem **String** e **classes wrappers como Integer e Double**;
- Classes imutáveis são particularmente úteis em programação multithread, pois eliminam a necessidade de sincronização;



Encapsulamento de Arrays

- Encapsulamento é a prática de **restringir o acesso direto aos atributos de uma classe**, expondo-os apenas através de métodos controlados (getters e setters);
- Quando trabalhamos com arrays ou coleções, o encapsulamento é igualmente importante;
- Arrays e coleções contêm múltiplos dados, e expô-los diretamente pode permitir modificações não controladas, causando inconsistências;
- Para encapsular arrays ou coleções, **usamos getters e setters**, garantindo que acessos e modificações sejam feitos de forma controlada;
- É importante fornecer cópias ao expor arrays ou coleções, **garantindo que o conteúdo original não seja alterado** diretamente;





Orientação a Objetos

Conclusão